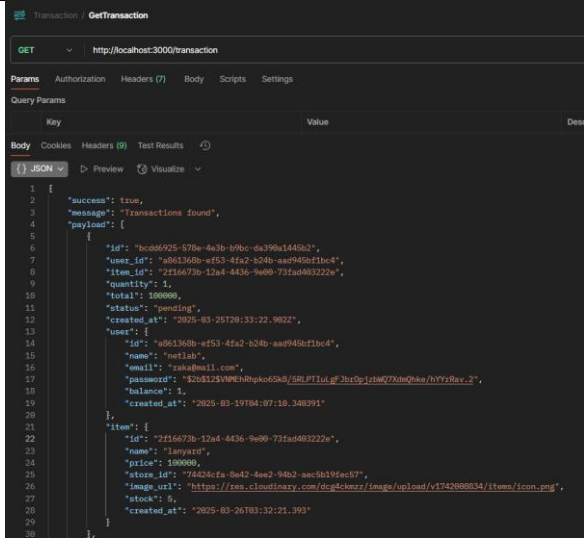
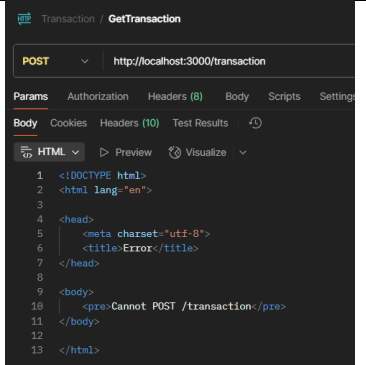


PRAKTIKUM SISTEM BASIS DATA

Nama	Raka Arrayan Muttaqien	No. Modul	6
NPM	2306161800	Tipe	TUTAM

1. Screenshoot Response

Method	endpoint	Success	Failed
GET	/transaction	 <pre> 1 { 2 "success": true, 3 "message": "Transactions found", 4 "payload": { 5 "id": "b0c6925-578e-4a3b-b9bc-da396a144b2", 6 "user_id": "a0613a0b-e553-41a2-b24b-aad940ef1bc4", 7 "item_id": "2f16673b-12a4-4436-9e09-731ad463222a", 8 "quantity": 1, 9 "total": 190000, 10 "status": "pending", 11 "created_at": "2025-03-26T03:33:22.902Z", 12 "user": { 13 "id": "a0613a0b-e553-41a2-b24b-aad940ef1bc4", 14 "name": "netlab", 15 "email": "raka@mail.com", 16 "password": "12u8125VMENhpk65k9/5RLPTiul.g?3hrOpj4t6Q7Xm@kue/hYyRav.2", 17 "balance": 1, 18 "created_at": "2025-03-19T04:07:10.348391" 19 }, 20 "item": { 21 "id": "2f16673b-12a4-4436-9e09-731ad463222a", 22 "name": "lampard", 23 "price": 190000, 24 "store_id": "74428cfa-8e42-4ae2-94b2-aec5b19ec57", 25 "image_url": "https://res.cloudinary.com/dca4-kmrz/image/upload/v1742968834/items/icon.png", 26 "stock": 5, 27 "created_at": "2025-03-26T03:32:21.393" 28 } 29 } 30 } </pre>	 <pre> 1 <!DOCTYPE html> 2 <html lang="en"> 3 4 <head> 5 <meta charset="utf-8"> 6 <title>Error</title> 7 </head> 8 9 <body> 10 <pre>Cannot POST /transaction</pre> 11 </body> 12 13 </html> </pre>

2. scalability, security, dan error handling

Pada post

Type	Implementasi	Deskripsi
Error Handling	<pre> } catch (error) { baseResponse(res, false, 500, error.message "Server error", null); } </pre>	Digunakan hampir di semua endpoint Tujuannya untuk menangkap kesalahan asinkron saat akses database atau logika lainnya yang mungkin gagal.
	<pre> if (!uuidRegex.test(store_id)) { return res.status(400).json({ success: false, message: "Invalid store_id format", payload: null }); } </pre>	Pengecekan format UUID untuk memastikan ID valid, menghindari SQL error, dan mencegah eksekusi dengan data tidak sah.
	<pre> if (!isUUID(id)) { return baseResponse(res, false, 400, "Invalid user ID format", null); } </pre>	
	<pre> if (quantity <= 0) { return res.status(400).json({ success: false, message: "Quantity must be larger than 0", payload: null }); } </pre>	Validasi jumlah transaksi untuk memastikan quantity lebih besar dari 0, menghindari kesalahan logika dalam proses.
	<pre> exports.getUserByEmail = async (req, res) => { const { email } = req.params; try { const user = await userRepository.getUserByEmail(email); if (!user) { return baseResponse(res, false, 404, "User not found", null); } } } </pre>	menangani error dengan cara menangkap kesalahan saat mengambil data user, lalu mengirim respons yang sesuai. Jika user tidak ditemukan, balas dengan 404. Jika terjadi error lain (seperti masalah server), balas dengan 500. Ini mencegah aplikasi crash dan

	<pre> return baseResponse(res, true, 200, "User found", user); } catch (error) { console.error("Error fetching user:", error); return baseResponse(res, false, 500, "Server error", null); } }; </pre>	memberikan info yang jelas.
	<pre> if (!email !password !name) { return baseResponse(res, false, 400, "Missing email, password, or name", null); } </pre>	Digunakan untuk menangani error input dari user. Jika email, password, atau name tidak diisi, maka langsung diberi respons 400 (Bad Request). Ini mencegah data kosong masuk ke sistem
Security	<pre> const isMatch = await bcrypt.compare(password, user.password); </pre>	untuk mengecek keaslian password tanpa harus menyimpan password asli. Dengan menggunakan bcrypt.compare, sistem bisa membandingkan password input dengan hash yang disimpan secara aman, tanpa harus tahu password aslinya. Ini sangat penting karena jika database bocor, password pengguna tetap aman karena sudah di-hash, bukan disimpan dalam bentuk teks biasa
	<pre> const emailRegex = /^[^\\s@]+@[^\\s@]+\\.\\.[^\\s@]+\$/; const passwordRegex = /^(?=.*[a- z])(?=.*[A-Z])(?=.*\\d)[A-Za-z\\d]{8,}\$/; </pre>	Fungsi dari regex email dan password ini adalah untuk memastikan input pengguna

		sudah benar dan aman sejak awal. Email dicek agar sesuai format yang valid, sedangkan password dicek agar kuat, minimal 8 karakter dan punya huruf besar, huruf kecil, serta angka. Untuk membantu mencegah data palsu dan meningkatkan keamanan akun pengguna
	<pre>const uuidRegex = /^[0-9a-f]{8}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{12}\$/i;</pre>	memastikan bahwa hanya UUID dengan format yang benar yang diproses sistem, sehingga membantu mencegah manipulasi data atau injection yang bisa terjadi jika format ID dibuat sembarangan oleh pengguna
	<pre>const cors = require("cors"); app.use(cors({ origin: 'https://os.netlabdte.com', methods: ['GET', 'POST', 'PUT', 'DELETE'] }));</pre>	Menggunakan Cross-Origin Resource Sharing (CORS) pada server Express.js, yaitu menentukan domain mana saja yang diizinkan untuk mengakses resource dari server.

Scalability	controllers JS item.controller.js JS store.controllers.js JS transaction.controller.js JS upload.controller.js JS user.controller.js database JS pg.database.js repositories JS item.repository.js JS store.repository.js JS transaction.repository... JS user.repository.js	Struktur folder modular (controller dan repository terpisah) yang berguna untuk penambahan fitur, pengujian, dan pengelolaan skala besar lebih efisien.
	<pre>require("dotenv").config() const { Pool } = require("pg") const pool = new Pool({ connectionString: process.env.PG_CONNECTION_STRING, ssl:{ rejectUnauthorized:false, }, }) const connect = async () => { try { await pool.connect() console.log("Connected to the database") } catch (error) { console.log(error) } } connect(); const query = async (text, params) => { try { const res = await pool.query(text, params) return res } catch (error) { console.log(error) } }</pre>	untuk mengatur koneksi database PostgreSQL secara efisien, sehingga bisa melayani banyak permintaan (query) secara bersamaan. Dengan menggunakan Pool, sistem tidak perlu membuat koneksi baru setiap kali ada permintaan, jadi lebih cepat

	<pre> } } module.exports = { query } </pre>	
	<pre> const baseResponse = (res, success, status, message, payload) => { return res.status(status).json({ success, message, payload }); }; </pre>	Digunakan di hampir seluruh controller untuk standarisasi response dari server. Ini membuat codebase lebih scalable seiring pertumbuhan jumlah endpoint.
	<pre> exports.getAllStores = async (req, res) => { try { const stores = await storeRepository.getAllStores(); baseResponse(res, true, 200, "stores retrieved successfully", stores); } catch (error) { baseResponse(res, false, 500, "error retrieving stores", error); } }; </pre>	Penggunaan async/await pada semua operasi asinkron terhadap database Untuk Memastikan performa tetap baik saat menangani banyak permintaan secara paralel, seperti pada getAllStores, loginUser, createItem, dll.

PART 2

1. Melakukan Deploy

a) Docker file

```
# Gunakan image Node.js
FROM node:20.15.1

# Set direktori kerja di dalam container
WORKDIR /app

# Salin file package.json dan package-lock.json terlebih dahulu
COPY package*.json ./

# Install dependencies terlebih dahulu
RUN npm install

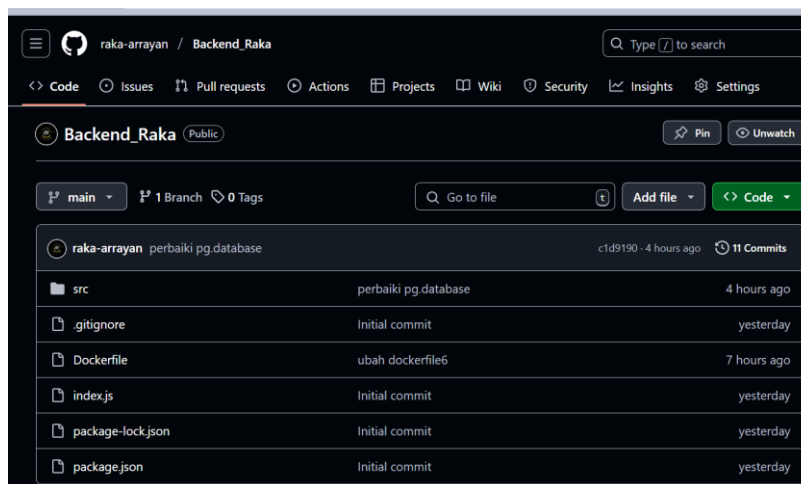
# Install dotenv jika belum
RUN npm install dotenv

# Salin seluruh source code ke dalam container
COPY . .

#Expose port yang digunakan oleh Express.js
EXPOSE 3000

# Perintah untuk menjalankan aplikasi
CMD ["node", "index.js"]
```

b) Push project



Stress Test

Screenshoot dan pdf juga sudah dilampirkan

